1. KUBERNETES

Kubernetes [1] (often abbreviated as K8s) is an open-source container orchestration platform developed by Google. It automates the deployment, scaling, and management of containerized applications, allowing developers and operators to efficiently run workloads across clusters of machines. In modern software development, Kubernetes has become a cornerstone for DevOps and cloud-native practices, helping organizations deliver applications faster, more reliably, and with greater flexibility.

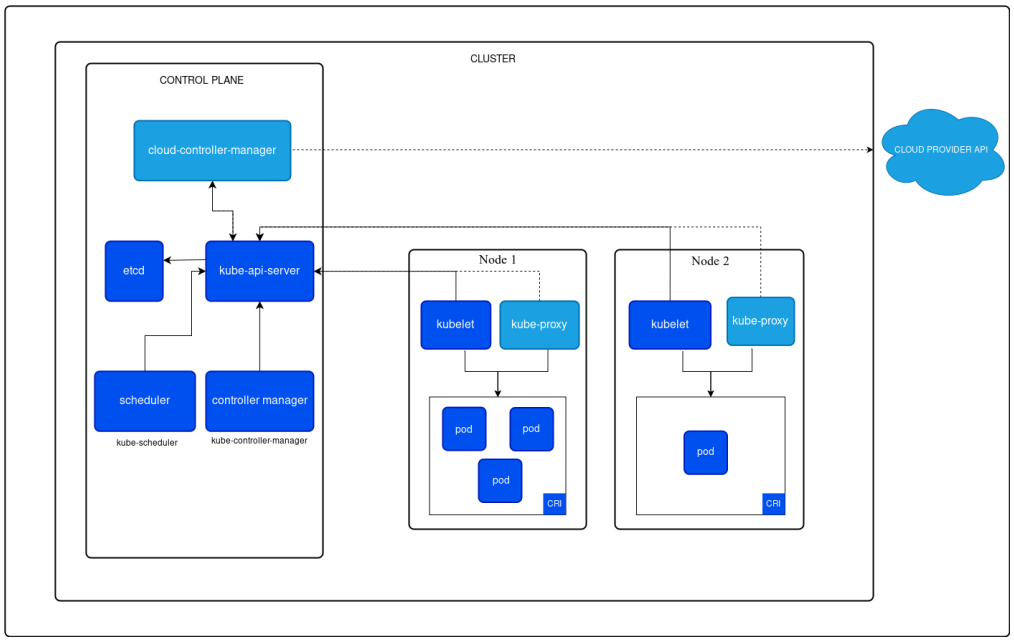


Figure 1: Kubernetes cluster architecture.

To set up a Kubernetes cluster, at least two machines are required: one acting as the master node and the other as a worker node. The master node should be more powerful than the other machines since it also takes role as a worker in this setup. For the sake of simplicity, we omit the configuration of Ingress services [2] – components that manage routing external traffic into the cluster, since this course focuses on interactions within the local network. The following steps are for configuring master and worker in Linux environment using Ubuntu Server 24.04 LTS.

1.1 Master

First, we need to disable the swap region. This can be done by opening the **/etc/fstab** file with sudo privileges and commenting out the line that defines the swap partition.

```
#/swap.img      none    swap    sw      0       0
```

[1] Kubernete cluster: https://kubernetes.io/docs/concepts/architecture/
[2] Ingress service setup: https://viblo.asia/p/k8s-basic-kubernetes-ingress-pgjLNb39L32.

If SELinux is installed and running, disable it by openning the **/etc/selinux/config** file and change the value of SELINUX key as follows.

```
SELINUX=disabled
```

For nodes in a Kubernetes cluster to communicate with each other, certain firewall ports must be opened. However, to simplify the setup in this course, we can disable the firewall entirely instead of configuring individual ports.

```
sudo ufw disable
sudo systemctl disable ufw.service
```

Optionally, we can redefine the hostname of the master node to simplify the configuration process later on. But this name must be consistent through all configuration.

```
hostnamectl set-hostname <MASTER_NAME>
```

Then open the **/etc/hosts** file and append the following lines to map the hostname with the corresponding IP address. IP of each individual machine can be retrieved by using the “hostname -I” command.

```
<MASTER_NAME> <IP_MASTER>
<WORKER_NAME> <IP_WORKER>
```

To configure networking for the Kubernetes cluster, create the **/etc/sysctl.d/k8s.conf** and **/etc/modules-load.d/k8s.conf** files, then add the following lines.

```
# for the /etc/modules-load.d/k8s.conf file
overlay
br_netfilter

# for the /etc/sysctl.d/k8s.conf file
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-ip6tables = 1
```

To install Kubernetes and the container runtime, we first need to add the official repository GPG key to the system keyring.

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.34/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.34/deb/ /" | \
  sudo tee /etc/apt/sources.list.d/kubernetes.list

sudo apt update -y
```

We will install and configure the Kubernetes cluster as the **root user**. Although running processes as root is generally not recommended, we will do so here for the sake of simplicity in the learning environment. First, using the “sudo su” command to switch to root user. Then follows the instructions below.

```
# install Containerd runtime
sudo apt install containerd.io -y
containerd config default > /etc/containerd/config.toml

# enable containerd to start at every reboot
sudo systemctl enable containerd
```

For Kubernetes cluster to connect with Containerd, change the key value in the **/etc/containerd/config.toml** file by the following values:

```
sandbox_image = "registry.k8s.io/pause:3.10.1"
SystemCgroup = true
```

Next, install the necessary Kubernetes tools required to set up the cluster. Once the installation command has completed successfully, **reboot** the system.

```
sudo apt install kubelet kubeadm kubectl -y
sudo systemctl enable kubelet
```

Once the machine has restarted, initialize the Kubernetes control plane.

```
kubeadm init --control-plane-endpoint=<MASTER_NAME>
```

After the initialization process is complete, Kubernetes will display a token for adding additional master nodes (in a multi-master deployment) as well as a separate token for joining worker nodes. Both commands follow the “kubeadm join” pattern, which must be executed on the respective nodes to connect them to the cluster.

```
You can now join any number of control-plane nodes by copying certificate authorities
and service account keys on each node and then running the following as root:

  kubeadm join master:6443 --token aqxvwu.5m4lvz2sr79110pu \
        --discovery-token-ca-cert-hash sha256:1959f015ab282288c24145ded6fd5c6e8d835663c56f92b09c6434e084349815 \
        --control-plane

Then you can join any number of worker nodes by running the following on each as root:

kubeadm join master:6443 --token aqxvwu.5m4lvz2sr79110pu \
        --discovery-token-ca-cert-hash sha256:1959f015ab282288c24145ded6fd5c6e8d835663c56f92b09c6434e084349815
```

Figure 2: Tokens for joining masters (above) and workers (below).

The respective join commands should be copied and saved for later use. However, if it is not saved, we can always retrieve the worker join command at any time by running the following command on the master node.

```
kubeadm token create --print-join-command
```

Next, copy the admin configuration file to the default location so that it can be accessed without restriction. This ensures that the kubectl command-line tool can communicate with the cluster without requiring additional configuration each time.

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
```

Since this setup is intended for educational purposes, we will configure the master node to also act as a worker node. However, this practice is not recommended in large-scale or production deployments, where master nodes should be dedicated exclusively to control-plane operations to ensure stability and security.

```
# setup master node for both master and worker jobs
kubectl taint nodes <MASTER_NAME> node-role.kubernetes.io/control-plane:NoSchedule-
```

At this stage, you should wait until all worker nodes have successfully joined the cluster. Once the cluster is fully formed, the next step is to install a Container Network Interface (CNI) plugin to enable communication between pods across different nodes. In this guide, we will use Calico as the networking solution.

```
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.26.3/manifests/calico.yaml

# optional: Label the worker node roles
kubectl label node <WORKER_NAME> node-role.kubernetes.io/worker=worker
```

Finally, wait until all cluster components are fully operational. To verify this, run the following two commands to check the current status of nodes and system pods. When

the setup is complete, the output should indicate that all nodes are in the “Ready” state and all pods are running successfully, as shown in the example below.

```
kube-system   calico-kube-controllers-755986996d-2n9ws   1/1   Running
kube-system   calico-node-jsnjj                          1/1   Running
kube-system   calico-node-pzmfm                          1/1   Running
kube-system   coredns-66bc5c9577-67x8k                   1/1   Running
kube-system   coredns-66bc5c9577-vfg6n                   1/1   Running
```

Figure 3: CMD: kubectl get pods --all-namespaces.

```
root@master:/home/hpcc# kubectl get nodes
NAME     STATUS   ROLES           AGE   VERSION
master   Ready    control-plane   57m   v1.34.1
worker   Ready    <none>          10m   v1.34.1
```

Figure 4: CMD: kubectl get nodes.

1.2 Worker

For installing Kubernetes on a worker node, the steps are almost identical to those performed on the master node. The main difference is that each worker node should be assigned a unique hostname to avoid conflicts within the cluster.

Turn off swap region in the **/etc/fstab** file.

```
# /dev/mapper/rl-swap
```

Turn off SELinux (if it is installed) in the **/etc/selinux/config** file.

```
SELINUX=disabled
```

Change hostname of worker nodes through the command below.

```
hostnamectl set-hostname worker
```

Disable system firewall through the command below.

```
sudo ufw disable
sudo systemctl disable ufw.service
```

Append hostname to IP mapping in the **/etc/hosts** file.

```
master <IP_MASTER>
worker <IP_WORKER>
```

Create the **/etc/modules-load.d/k8s.conf** and **/etc/sysctl.d/k8s.conf** files as below.

```
# for the /etc/modules-load.d/k8s.conf file
overlay
br_netfilter

# for the /etc/sysctl.d/k8s.conf file
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-ip6tables = 1
```

Add the repository of Containerd and Kubernetes to system package.

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo mkdir -p /etc/apt/keyrings

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.34/deb/Release.key | \
  sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.34/deb/ /" | \
  sudo tee /etc/apt/sources.list.d/kubernetes.list

sudo apt update
```

Install and enable Containerd by using the commands below.

```
# install Containerd runtime
sudo apt install containerd.io -y
containerd config default > /etc/containerd/config.toml

# enable containerd to start at every reboot
sudo systemctl enable containerd
```

Change the corresponding key value in the **/etc/containerd/config.toml** file.

```
sandbox_image = "registry.k8s.io/pause:3.10.1"
SystemCgroup = true
```

Install and enable Kubernetes service using the commands below.

```
sudo apt install kubelet kubeadm kubectl -y
sudo systemctl enable kubelet
```

After the installation is complete, **reboot** the system. Once the machine has restarted, join the worker node to the cluster by executing the "kubeadm join command" that was provided during the control-plane initialization, as shown in Figure 2.

1.3 Testing Kubernetes

To verify that Kubernetes has been installed successfully, you can deploy a simple web application using the Nginx image. Run the following commands to create a deployment and expose it as a service.

```
kubectl create deployment web-app --image nginx --replicas 2
kubectl expose deployment web-app --type NodePort --port 80
```

To verify where the web application is deployed, run the command as shown in Figure 6, the Nginx web app is running across both the master and worker nodes since the replica count is set to 2. Figure 5 demonstrates that the service has been exposed on port 30553, which is mapped to port 80 inside the container. This mapping enables external clients to access the application through the specified NodePort.

```
root@master:/home/hpcc# kubectl get svc web-app
NAME      TYPE       CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
web-app   NodePort   10.106.214.73   <none>        80:30553/TCP   3h48m
```

Figure 5: CMD: kubectl get svc web-app.

```
root@master:/home/hpcc# kubectl get pods -o wide
NAME                       READY   STATUS    RESTARTS   AGE     IP               NODE
 GATES
web-app-c8b4f854b-cnh8s    1/1     Running   0          3h50m   192.168.171.68   worker
web-app-c8b4f854b-rmgd7 _  1/1     Running   0          3h50m   192.168.219.65   master
```

Figure 6: CMD: kubectl get pods -o wide.

In this demonstration, the master node has the IP of "172.28.13.113", hence the URL is "172.28.13.113:30553" and the browser should appear as in Figure 7.

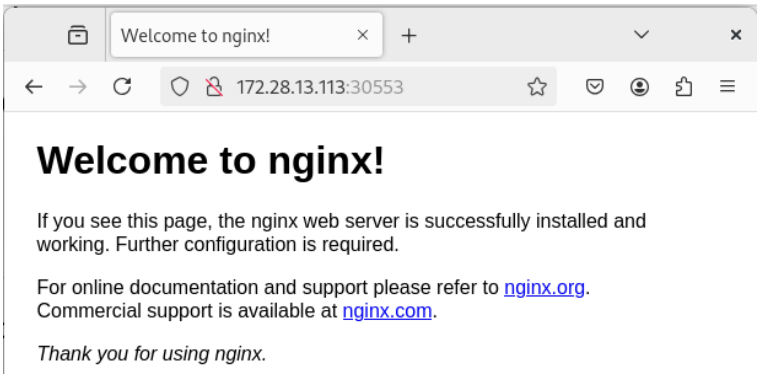


Figure 7: Nginx homepage is on indicating that it is successfully deployed.

2. APACHE KAFKA

2.1 Overview of Apache Kafka

Apache Kafka is one of the most popular distributed event streaming platforms, which is widely used for building real-time data pipelines and streaming applications. Kafka is designed to handle massive volumes of data in a scalable and fault-tolerant manner, making it ideal for use cases like real-time analytics, data ingestion, and event-driven architectures. Kafka follows a distributed publish-subscribe messaging model, where data is organized into topics. These topics can be partitioned to enable parallel processing across multiple consumers within a same group, and replicated across brokers to ensure high availability and fault tolerance.

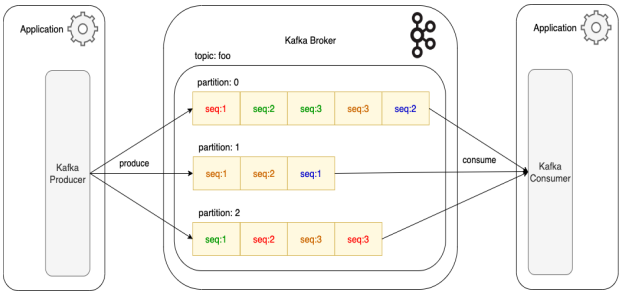


Figure 8: Group contains only one consumer.

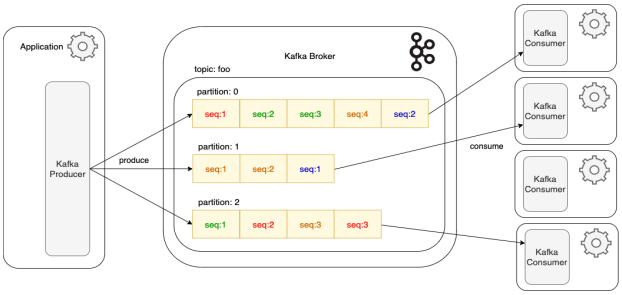


Figure 9: Group contains multiple consumers.

Kafka organizes data into topics, each of which may have multiple partitions where the actual data lies. A Kafka Producer operates independently and specifies the topic it wants to write to. The data records are then distributed across the topic partitions based on the producer partitioning strategy. In contrast, a Kafka Consumer operates as part of a consumer group. For example, as shown in Figure 8, if a consumer group contains only one consumer, it will receive data from all partitions. Meanwhile, if the group contains multiple consumers as in Figure 9, Kafka will assign different partitions to each consumer, enabling parallel processing for higher throughput.

2.2 Deploy Apache Kafka with Kubernetes

In Kubernetes, a StorageClass acts as a template that tells the cluster how and where to create persistent storage when an application requests it. With a StorageClass, Kubernetes automatically provisions volumes based on the defined backend storage provider. The Local Path Provisioner (from Rancher) provides a simple way to use each node's local filesystem as storage, automatically creating and deleting local directories when PersistentVolumeClaims are made or released. This makes it especially useful in lab or development environments, where fast, lightweight, and node-local storage is sufficient – offering easy setup, automatic cleanup, and no dependency on external systems.

```
kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-provisioner/master/deploy/local-path-storage.yaml
```

First, we create a dedicated namespace for Kafka to logically isolate its components and manage resources more efficiently within the Kubernetes cluster.

```
# create kafka namespace
kubectl create namespace kafka
```

The following YAML file is used to deploy Apache Kafka with two brokers in Kubernetes. The YAML file consists of three main components:

- A **ConfigMap** to store Kafka base configuration (base.properties), enabling Kafka pods to load these settings at startup.
- A headless **Service** (with clusterIP: None), which creates the stable DNS entries for each pod. This is essential because pod IPs can change when pods are recreated, making direct IP-based communication unreliable. With headless service, the DNS format follows a specific predefined pattern, allowing each Kafka broker to communicate with the others regardless of IP changes.
- A **StatefulSet** to manage the Kafka brokers. StatefulSets provide stable pod identities and persistent storage, which are critical for Kafka operation, ensuring each broker retains its identity and message log data even after restarts.

```
# file: kafka.yaml
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: kafka-config
  namespace: kafka
data:
  base.properties: |
    process.roles=broker,controller
    controller.listener.names=CONTROLLER
    listeners=PLAINTEXT://:9092,CONTROLLER://:9093
    inter.broker.listener.name=PLAINTEXT
    log.dirs=/var/lib/kafka/data
    controller.quorum.bootstrap.servers=kafka-0.kafka-service.kafka.svc.cluster.local:9093,kafka-1.kafka-service.kafka.svc.cluster.local:9093
    controller.quorum.voters=0@kafka-0.kafka-service.kafka.svc.cluster.local:9093,1@kafka-1.kafka-service.kafka.svc.cluster.local:9093
---
apiVersion: v1
kind: Service
metadata:
  name: kafka-service
  namespace: kafka
spec:
  ports:
  - name: kafka
    port: 9092
    protocol: TCP
    targetPort: 9092
  - name: controller
    port: 9093
    protocol: TCP
    targetPort: 9093
  clusterIP: None
  selector:
    app: kafka
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: kafka
  namespace: kafka
spec:
  serviceName: kafka-service
  replicas: 2
  podManagementPolicy: Parallel
  selector:
  matchLabels:
    app: kafka
  template:
  metadata:
    labels:
      app: kafka
  spec:
```

```
    containers:
      - name: kafka
        image: apache/kafka:latest
        command: [ "sh", "-c" ]
        args:
        - |
          NODE_ID=$(hostname | awk -F'-' '{print $2}');
          CLUSTER_ID="uhaGgVSiR3WO5yPkn_pKgw";
          echo "node.id=$NODE_ID" >> /opt/kafka/config/server.properties;
          echo "advertised.listeners=PLAINTEXT://kafka-$NODE_ID.kafka-service.kafka.svc.cluster.local:9092,CONTROLLER://kafka-$NODE_ID.kafka-service
      .kafka.svc.cluster.local:9093" >> /opt/kafka/config/server.properties;
          cat /kafka/config/base.properties >> /opt/kafka/config/server.properties;
          /opt/kafka/bin/kafka-storage.sh format -t $CLUSTER_ID -c /opt/kafka/config/server.properties
          /opt/kafka/bin/kafka-server-start.sh /opt/kafka/config/server.properties
        ports:
        - containerPort: 9092
        - containerPort: 9093
        volumeMounts:
        - name: datadir
          mountPath: /var/lib/kafka/data
        - name: config
          mountPath: /kafka/config
    volumes:
    - name: config
      configMap:
        name: kafka-config
  volumeClaimTemplates:
  - metadata:
        name: datadir
    spec:
        accessModes: ["ReadWriteOnce"]
      storageClassName: local-path
      resources:
        requests:
          storage: 10Gi
```

The commands below demonstrate how to deploy the aforementioned YAML file to Kubernetes and how to completely delete the resources when no longer needed.

Note: In this exercise, each node serves as both a controller and a broker for simplicity. However, in practice, it is recommended to separate controller and broker roles.

```
# deploy kafka
kubectl apply -f <YAML_FILE>

# delete kafka completely
kubectl delete configmap <CONFIGMAP_NAME> -n <NAMESPACE>
kubectl delete service <SERVICE_NAME> -n <NAMESPACE>
kubectl delete statefulset <STATEFULSET_NAME> -n <NAMESPACE>
kubectl delete pvc -l app=<STATEFULSET_NAME> -n <NAMESPACE>
```

To scale Kafka horizontally, we simply increase the number of broker replicas by modifying the replicas value and update the corresponding quorum configuration incrementally, e.g. the quorum configurations for three brokers are illustrated below.

```
controller.quorum.bootstrap.servers=kafka-0.kafka-service.kafka.svc.cluster.local:9093,kafka-1.kafka-service.kafka.svc.cluster.local:9093,kafka-2.
      kafka-service.kafka.svc.cluster.local:9093
controller.quorum.voters=0@kafka-0.kafka-service.kafka.svc.cluster.local:9093,1@kafka-1.kafka-service.kafka.svc.cluster.local:9093,2@kafka-2.kafka-
      service.kafka.svc.cluster.local:9093
```

Then, after all Kafka broker pods have started successfully and are in the Running state, execute the following command to create a specific Kafka topic.

```
kubectl exec -it kafka-0 -n kafka -- /opt/kafka/bin/kafka-topics.sh --create --bootstrap-server localhost:9092 --topic <TOPIC_NAME>
```

Since Kafka is deployed using a StatefulSet with a headless service, each broker is assigned a stable DNS name but no external load-balanced IP, making it slightly tricky to determine the exact connection endpoint. To allow external applications (e.g., Kafka Consumer and Kafka Producer) to connect, we first need to retrieve the internal IP address of each Kafka pod corresponding to its pod name and the stable DNS name which is auto generated, as illustrated in Figure 10.

```
root@master:/home/hpcc/kafka# kubectl get pods -n kafka -o wide
NAME      READY   STATUS    RESTARTS   AGE     IP                NODE     NOMINATED NODE   READINESS GATES
kafka-0   1/1     Running   0          9m36s   192.168.219.122   master   <none>           <none>
kafka-1   1/1     Running   0          9m36s   192.168.171.122   worker   <none>           <none>
```

Figure 10: CMD: kubectl get pods -o wide -n kafka.

Next, we append each pod IP address, along with its corresponding hostname and StatefulSet DNS name, to the **/etc/hosts** file. This step ensures that Kafka Client can correctly resolve and communicate with each broker.

```
# inside the /etc/hosts file
<POD_KAFKA_0_IP> kafka-0 kafka-0.kafka-service.kafka.svc.cluster.local
<POD_KAFKA_1_IP> kafka-1 kafka-1.kafka-service.kafka.svc.cluster.local
```

Finally, we verify the deployment by running a simple producer–consumer test using the following code. The producer sends messages to a topic, while the consumer subscribes to the same topic and reads the messages to confirm end-to-end communication. The required Python library can be installed as commands below.

```
sudo apt install -y software-properties-common
sudo add-apt-repository ppa:deadsnakes/ppa -y
sudo apt update
sudo apt install python3.10 python3.10-venv

# should not run with root
python3.10 -m venv $HOME/dissys
source $HOME/dissys/bin/activate # active virtual environment, should be run before hand
pip install --upgrade pip
pip install confluent_kafka
```

Note: Students are advised to create and use a virtual environment in which Python is downgraded to version 3.10 to ensure compatibility with future components.

```
from confluent_kafka import Consumer, KafkaError

# use earliest to fetch all data and latest to fetch new data only
conf = { 'bootstrap.servers': 'kafka-0:9092,kafka-1:9092', 'group.id': 'test-group', 'auto.offset.reset': 'earliest' }

consumer = Consumer(conf)
consumer.subscribe(['<TOPIC_NAME>'])
try:
  while True:
    msg = consumer.poll(1.0)  # timeout in seconds
    if msg is None:
      continue
    if msg.error():
        print(f"Error: {msg.error()}")
        break
    print(f"Received message: {msg.value().decode('utf-8')} from {msg.topic()} [{msg.partition()}] at offset {msg.offset()}")
except KeyboardInterrupt:
  print("Stopping consumer")
finally:
  consumer.close()
```

```
from confluent_kafka import Producer

conf = { 'bootstrap.servers': 'kafka-0:9092,kafka-1:9092' }
producer = Producer(conf)

def delivery_report(err, msg):
  if err is not None:
    print(f"Delivery failed: {err}")
  else:
    print(f"Message delivered to {msg.topic()} [{msg.partition()}]")

for i in range(10):
  message = f'Hello Kafka {i}'
  producer.produce('<TOPIC_NAME>', message.encode('utf-8'), callback=delivery_report)
  producer.poll(0)  # Trigger delivery callback

producer.flush()  # Wait for all messages to be delivered callback
```

3. EXERCISES

**** Submission:** Students should work collaboratively in groups to complete the assigned tasks. Each group must submit a **written report** and a **demonstration video** covering their implementation and results. **No presentation or live demonstration is required during the final lab session.**

Note: When a node's IP address changes in a Kubernetes cluster, the cluster may become unstable or stop functioning correctly. Students must successfully deploy Kubernetes and verify their implementation before preparing the report and demonstration video. The following commands show how to reset and start a fresh Kubernetes cluster. On each node, run:

```
sudo kubeadm reset -f
sudo rm -rf /etc/kubernetes
sudo rm -rf /var/lib/etcd
sudo rm -rf /var/lib/kubelet/*
sudo rm -rf /etc/cni/net.d
sudo rm -rf $HOME/.kube
```

Then, reboot the system and follow the previously described steps starting from the "kubeadm init –control-plane-endpoint=<MASTER_NAME>" command.